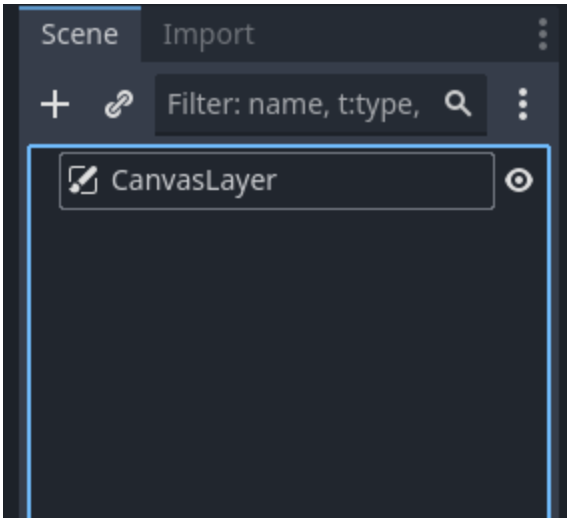
Basket Catch 4 - UI and Changing Scenes

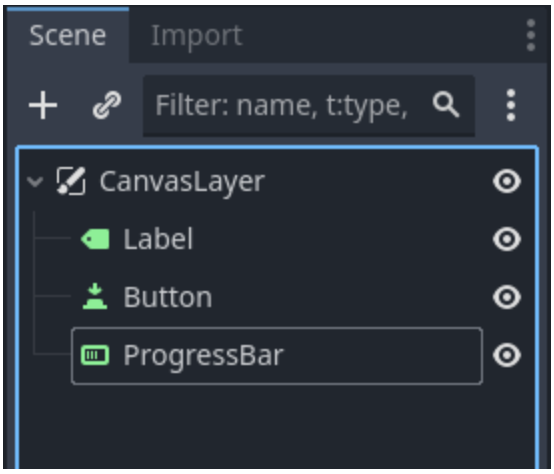
At this point, we have implemented nearly all of our major mechanics. Now it is time for us to consider the user experience of our game. One major component of UX (user experience) design is the User Interface (or UI). The UI of a game would include any information that we display on the screen as well as any menus or buttons. Godot puts all of these UI nodes in the Control category. Let's start by talking about the basic set up of a UI.

Anatomy of a UI Scene

We will make any new UI its own scene. This has a few benefits, the largest being that it will be reusable across several different scenes. To have our UI properly display we will make the root node of our UI scene a new Node: A CanvasLayer. This will make our UI elements display and interact more predictably. (Remember, we should always rename root nodes!)



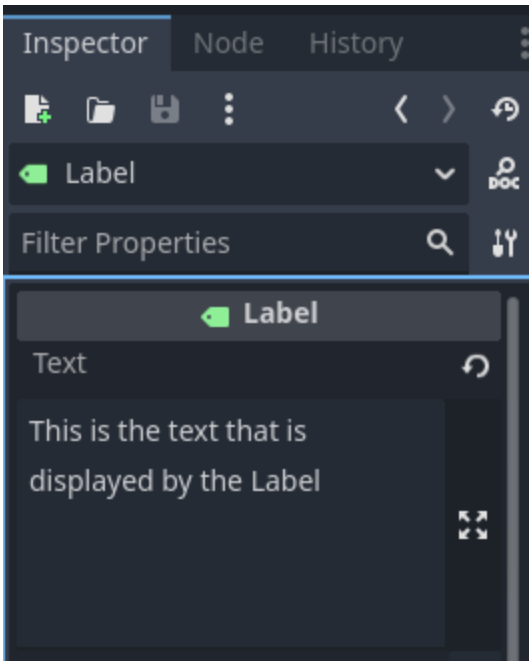
We will then attach UI components as *children* of the CanvasLayer:



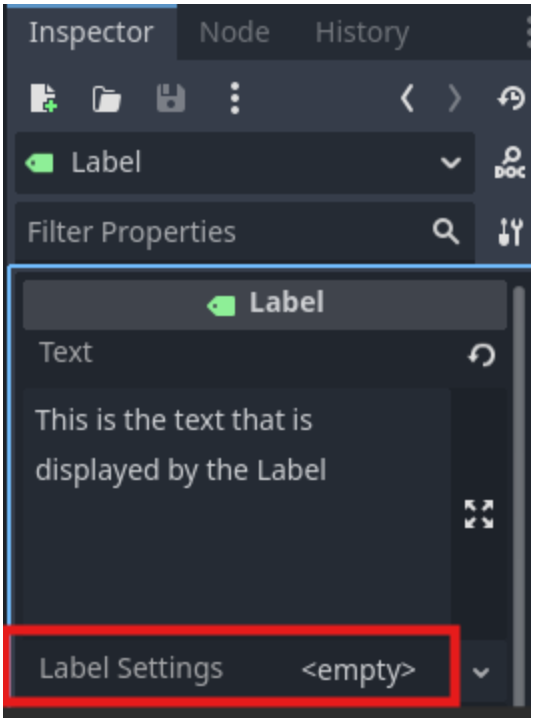
Let's talk about some of the most common Control Nodes.

Displaying Text: Label

The Label Node is used to display text on the screen. The text that is displayed by the Label is stored in the Label's *text* property. We can see this in the inspector:



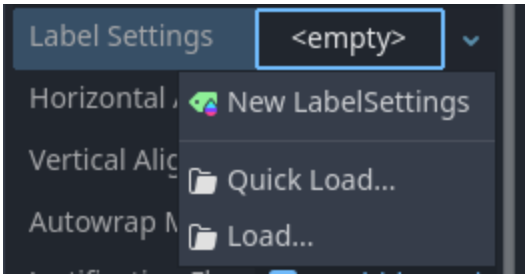
We can edit the way the text looks by changing the Label's *label_settings* property. This is most easily done through the inspector:



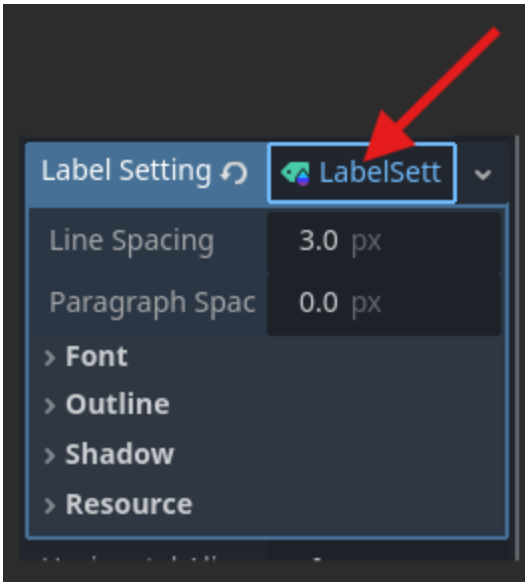
Label Settings

The label_settings property of a Label holds a resource called "LabelSettings". This resource allows us to change everything from the size of the font to the actual font itself. To add a new

Label Settings to a Label, we will first click the "<empty>" dropdown by the label_settings in the inspector. We will then choose: "New LabelSettings"



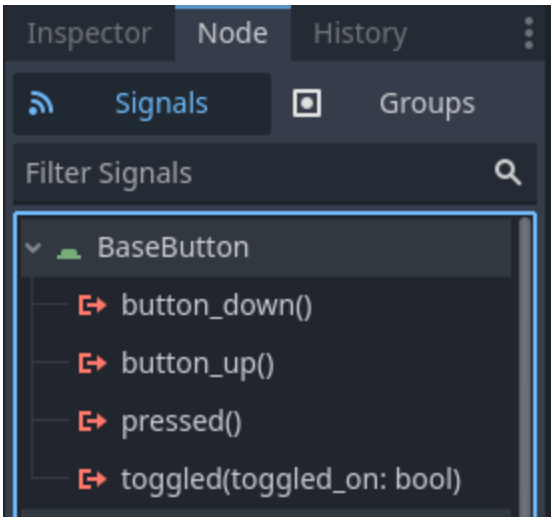
We can then open the LabelSettings by clicking on the LabelSetting itself:



Most of the things we would want to edit can be found under the "Font" category, but feel free to explore the other categories found here!

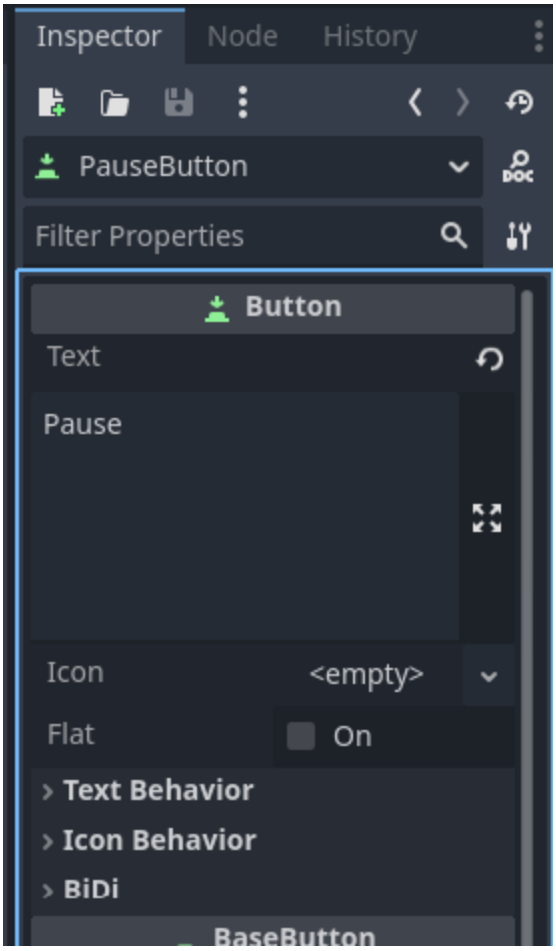
Interactive UI: Button

A Button allows us to run code when a player presses it. We can tell when a Button has been pressed by using its *signals* below is its signal list:



The most commonly used signal for Button is the *pressed()* signal. This signal is emitted whenever the Button is pressed. Remember, we need to connect the signal to a Node with a script attached to it for us to be able to do anything when the signal is emitted.

You'll also see that our Button has a *text* property. This is the text displayed on our Button. Here's an example of what a "Pause" Button's *text* property would look like:

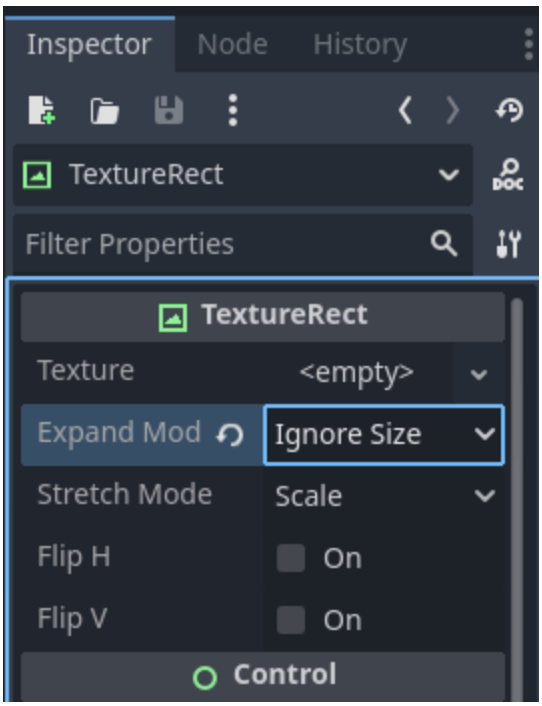


Adding Images to UI: TextureRect

TextureRect is very similar to Sprite2D in that we use both of them to display images. However, TextureRect is specifically used for UI. When using a TextureRect we may want it to appear as a background element of our UI. We can change which UI elements are drawn on top of others by changing the element's z_index property in the inspector. This can be found in the "Canvas Item" category:

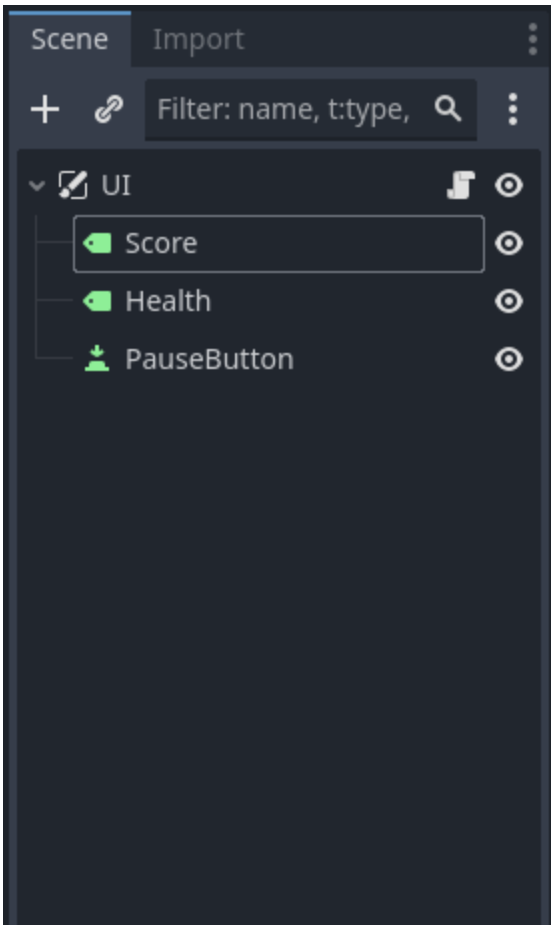
Elements with a *larger* z_index value will be drawn *on top* of elements with a lower z_index value.

When changing a TextureRect's size we may also want to change its expand_mode property to "Ignore Size" in the inspector. This will let us resize the TextureRect to a size *smaller* than its original resolution:



Adding Code to a UI

For our UI scenes we will always attach a script to the root Node (The CanvasLayer Node). We then control the other UI elements *with* that script by using Node references. A basic UI scene should look something like this when we've attached a script:



And here's what our script would look like once we've brought in all of our Node references:

```
extends CanvasLayer

@onready var score: Label = $Score
@onready var health: Label = $Health
@onready var pause_button: Button = $PauseButton


# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass
```

As stated before, some of our UI Nodes will send out signals (like Buttons), here's what our code would look like once we've connected the Button's "pressed" signal:

```
extends CanvasLayer

@onready var score: Label = $Score
@onready var health: Label = $Health
@onready var pause_button: Button = $PauseButton


# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass


func _on_pause_button_pressed() -> void:
    pass # Replace with function body.
```

Let's go over some common tasks that we would need to use code for

Changing Label text in code

Often, we may want to change the text being held in a Label when something happens in our game. We can do this by accessing a Label's *text* property using dot notation. An important thing to remember is that a Label's *text* property will only hold *Strings*. If we try to store a different data type, like an int or a bool, in the *text* property we will get an error. Let's start with the most simple application of just changing the text to something new. In this example, we want to change the score to show "0" when the level starts:

```
extends CanvasLayer

@onready var score: Label = $Score
@onready var health: Label = $Health
@onready var pause_button: Button = $PauseButton


# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    score.text = "0"
    pass # Replace with function body.
```

```
# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass


func _on_pause_button_pressed() -> void:
    pass # Replace with function body.
```

Notice that we need to put quotation marks around the 0. This tells Godot that we are giving the text property a String and not an int or float. However, we'll often want our score to change when our player does something that increases it. The best way to do this is to change our score Label's *text* property to display the data stored in an Autoload's *score* property (For more on creating Autoloads and declaring properties and methods() for an Autoload see Basket Catch 3 - Globals). However, the data stored in such a property would typically *not* be a String. Don't worry though, we can convert (or *cast*) any data type to a String using the str() method(). Here's what setting the score Label's text to our Autoload's score property every frame would look like:

```
extends CanvasLayer

@onready var score: Label = $Score
@onready var health: Label = $Health
@onready var pause_button: Button = $PauseButton


# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    score.text = "0"
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    score.text = str(Autoload.score)
    pass


func _on_pause_button_pressed() -> void:
    pass # Replace with function body.
```

If you were to do this you would see that the only thing that our score Label displays is a number. We may want our score Label to have other text, like "Score: " or "Current Score: ". To do this we can combine, or *concatenate*, Strings using the + operator. Here's how we would edit the script above to make the score Label display both "Score: " and the current score:

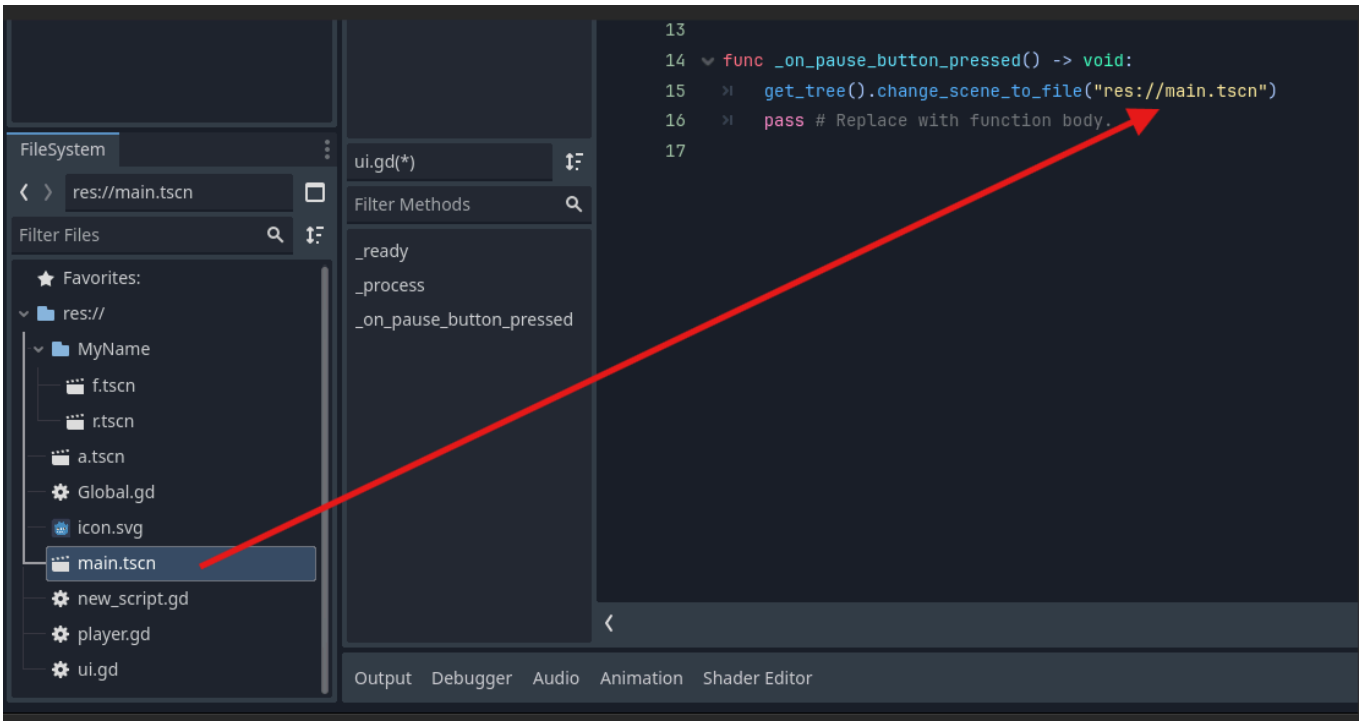
```
extends CanvasLayer

@onready var score: Label = $Score
@onready var health: Label = $Health
@onready var pause_button: Button = $PauseButton


# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    score.text = "0"
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    score.text = "Score: " + str(Autoload.score)
    pass


func _on_pause_button_pressed() -> void:
    pass # Replace with function body.
```

Changing Scenes

A common action we may take when a Button is pressed is to change the current scene. We may do this for "Start", "Restart", or "Continue" Buttons. To change a scene we will use the below code:

```
get_tree().change_scene_to_file()
```

We need to drag the scene that we would like to change to from the FileSystem and place it in the parentheses of the change_scene_to_file() method():



Now it's your turn!

In your Basket Catch project do the following

- Make the following scenes
  - Start Screen
  - End Screen
  - Player UI
- For the Start Screen
  - Add a background using a TextureRect
  - Add a Button that changes the scene to the game scene
  - Add a Label that displays the name of your game
- For the End Screen
  - Add a background using a TextureRect
  - Add a "Restart" Button that changes the scene to the game scene
  - Add a Label saying Game Over
  - Add a Label that displays the score
- For the Player UI
  - Add a Label that displays the current score (this should change when you collect something)
  - Place the Player UI scene in your Main scene
- In your Main Scene
  - Attach a script to your Root Node
  - Add a Timer Node to the scene

- Set the Timer's "autostart" property to true in the inspector
- Connect the Timer's "timeout()" signal to the Root Node
- Change the scene to the End Screen scene when the "timeout()" signal function runs